

- Shared Read locks
 - Conflicts
 - Scheduling based on operations performed
 - The common scheduler
 - Shared vs exclusive locks
 - Upgrade locks

Conflicts in a schedule

- Conflicts are a concept to capture the pairwise relationship of operations (and by extension transactions) accessing the same object in a schedule.
- Each operation writing an object has one conflict with each operation by a different transaction accessing this object.

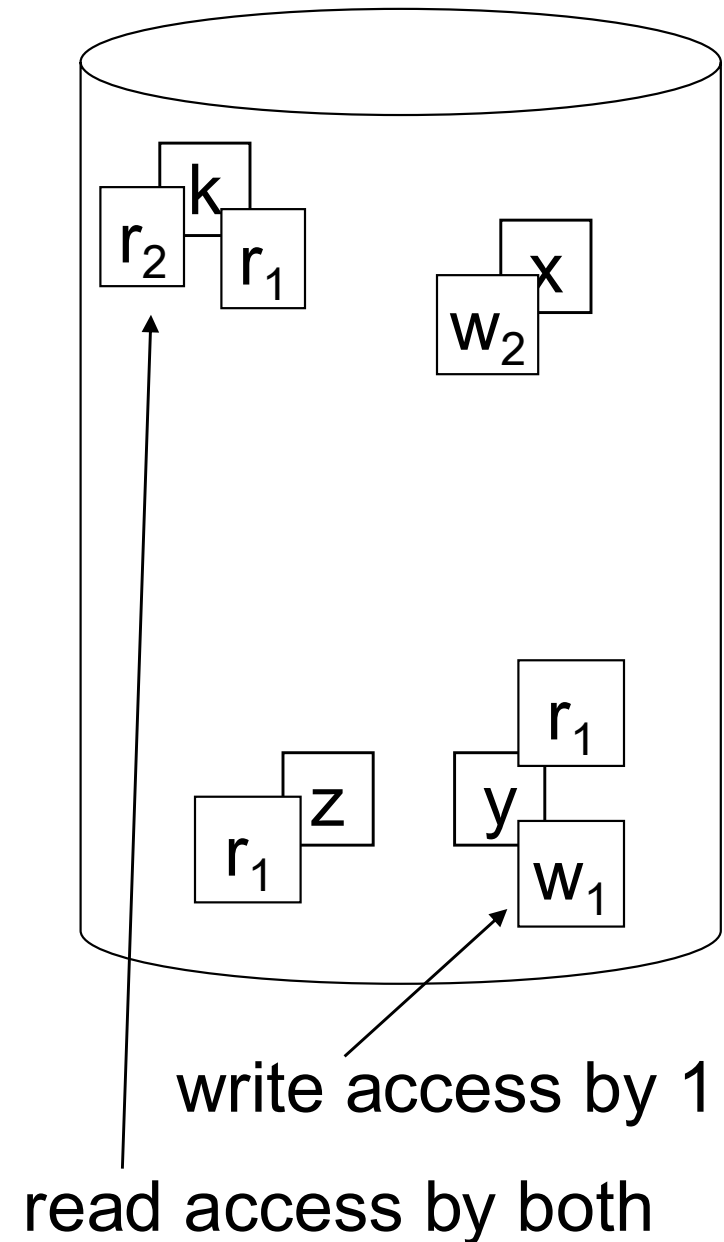
We write these conflicts for object x :

$r_1[x], r_2[x], c_2, w_1[x], c_1, r_3[x], w_3[x], c_3$

- As ordered pairs of operations:
- $r_2[x], w_1[x]$, and $w_1[x], r_3[x]$, and also $w_1[x], w_3[x]$,
- $r_1[x], w_3[x]$, and $r_2[x], w_3[x]$.
- Idea: order in which transactions commit should be compatible with this order.

Conflict-free Transactions

- A set of transactions is *conflict-free*, if it has no conflicts.
- Note that this is independent of the scheduling of the transactions.
- Note also that the transactions can still have shared objects and hence may not be disjoint-access.
- Intuition:
- Shared objects are from the view of these transactions constant:
- A set of conflict-free transactions should not be delayed either.



The simple scheduler is pessimistic

- The simple scheduler expects for every read, that it might be followed by a write:

- S: $r_1[x], r_1[y], w_1[x], c_1, r_2[x], w_2[x], c_2, r_3[x], w_3[z], c_3$
- TA1: $r_1[x], r_1[y], w_1[x], c_1$
- TA2: $r_2[x]$ _____, $w_2[x], c_2$
- TA3: $r_3[x]$ _____, $w_3[z], c_3$

transactions TA2 and TA3 waiting, although no conflict yet.

Oh, good that TA2 was waiting

Transaction TA3 waited, although it was not necessary!

Info for transaction scheduling

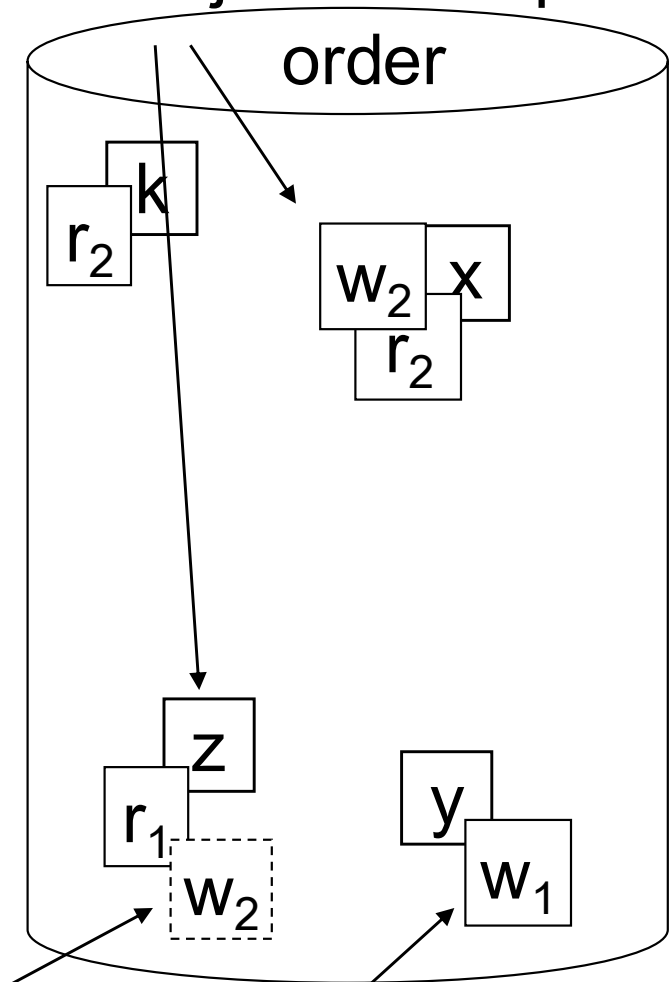
- Keep a record of which **open** transaction accessed (or tried to do so) which object with which operation. Tag object with:

r_2 • The operation performed (r, w)
• The transaction ID (the index)

- Scheduling strategies can be seen as deciding access based on the tags on x, whether a next operation on x can proceed or needs to be blocked in order to avoid harm.

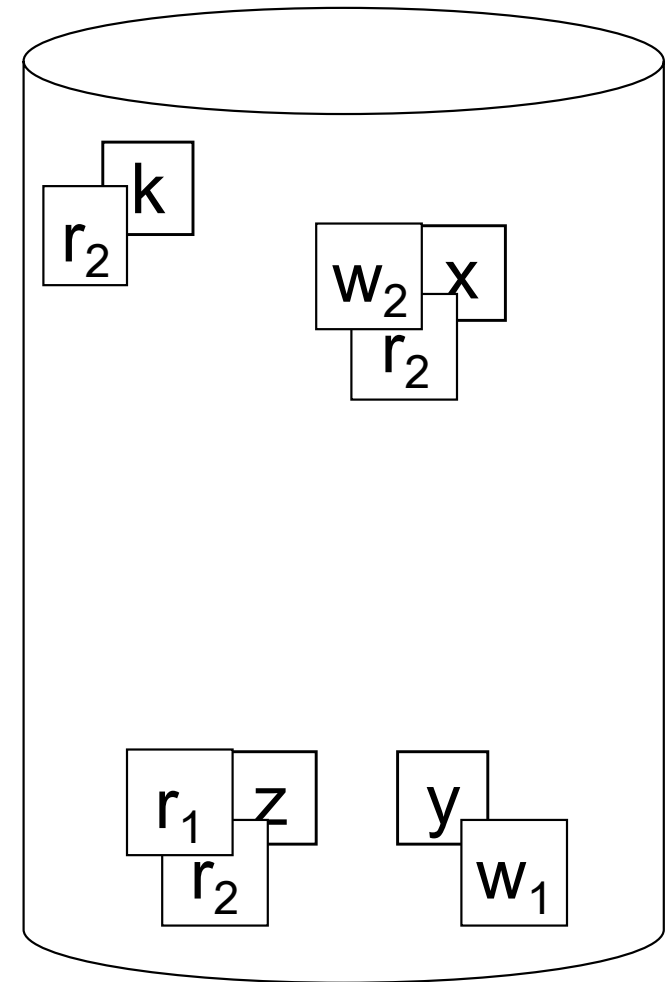
- Blocked operations with dotted lines

Stack all operations on object in temporal



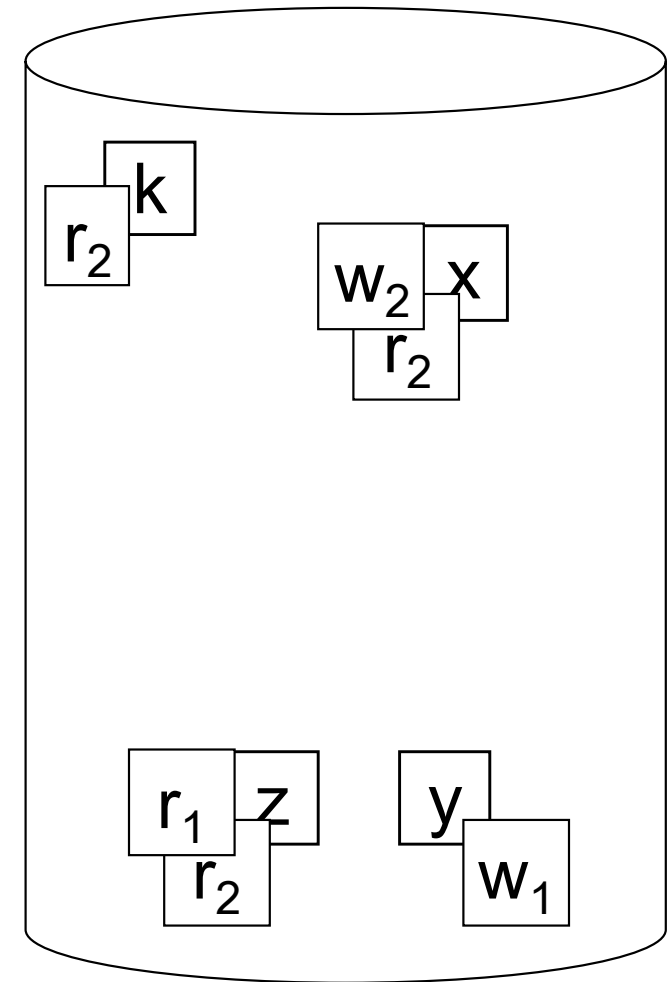
Reasoning over open transactions

- Extend definitions to open transactions as follows:
- The current set of partially executed open transactions:
- All operations from all open transactions already executed at this point in time, NOT including any currently blocked operations.
- Note, if they would all commit at this stage they would form a set of transactions as we already discussed it.



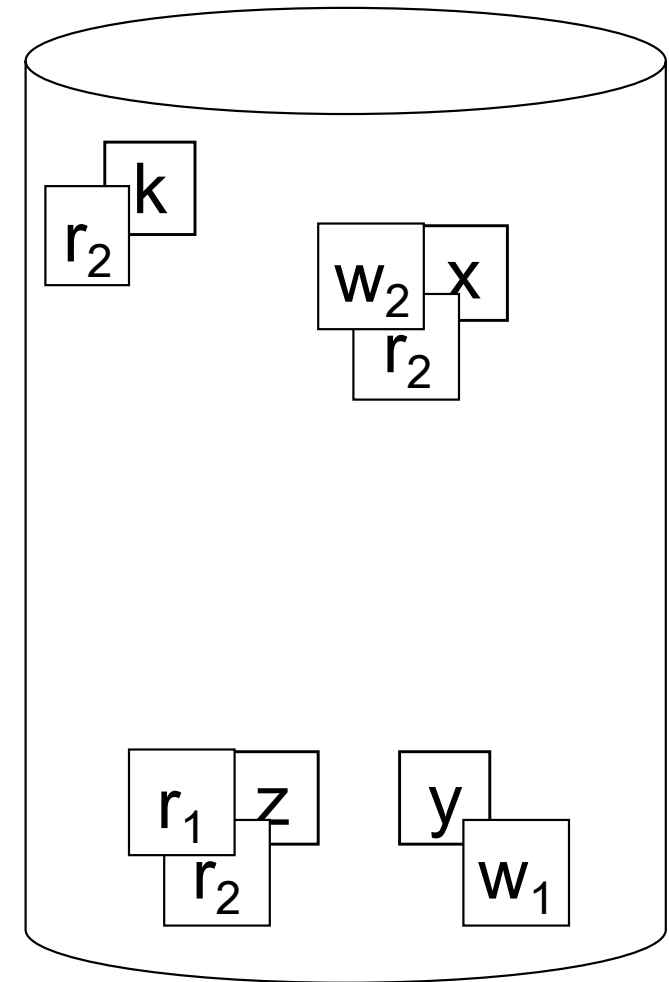
Declarative definition of schedulers

- Consider the current set of partially executed open transactions for each moment in time.
- **Simple** scheduler: Block any operation that would cause this set to be not **disjoint-access** any more.
- New scheduler:
- *Common scheduler*: Block any operation that would cause this set to be not **conflict-free** any more.



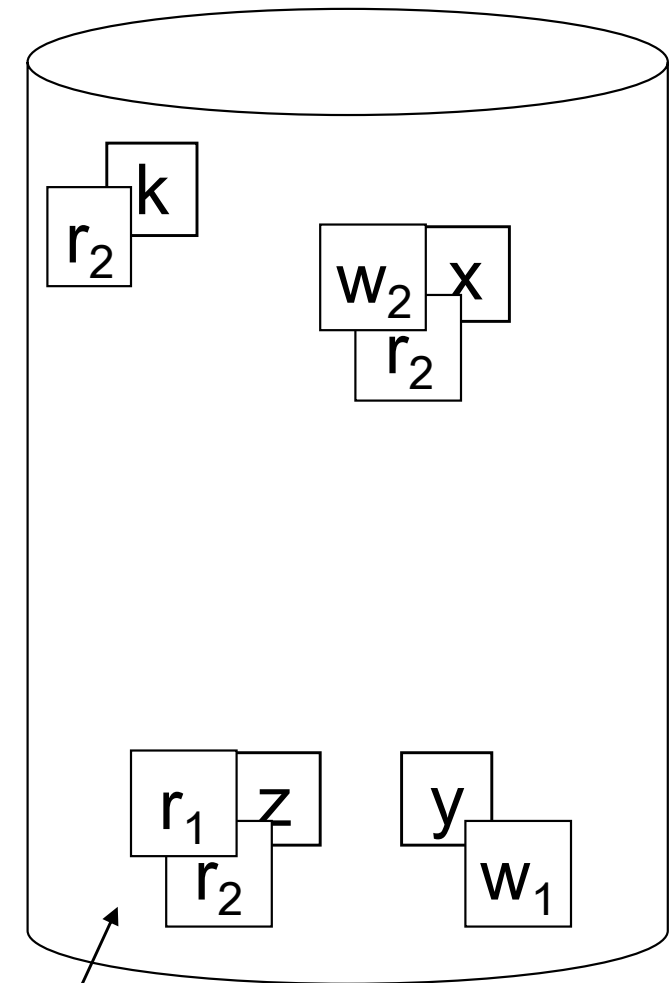
Declarative definition of schedulers

- Consider the current set of partially executed open transactions for each moment in time.
- **Simple** scheduler: Block any operation that would cause this set to have a **shared object**.
- **Common** scheduler: Block any operation that would cause this set to have a **conflict**.
- Can all operations on the right pass in common scheduler?



Declarative definition of schedulers

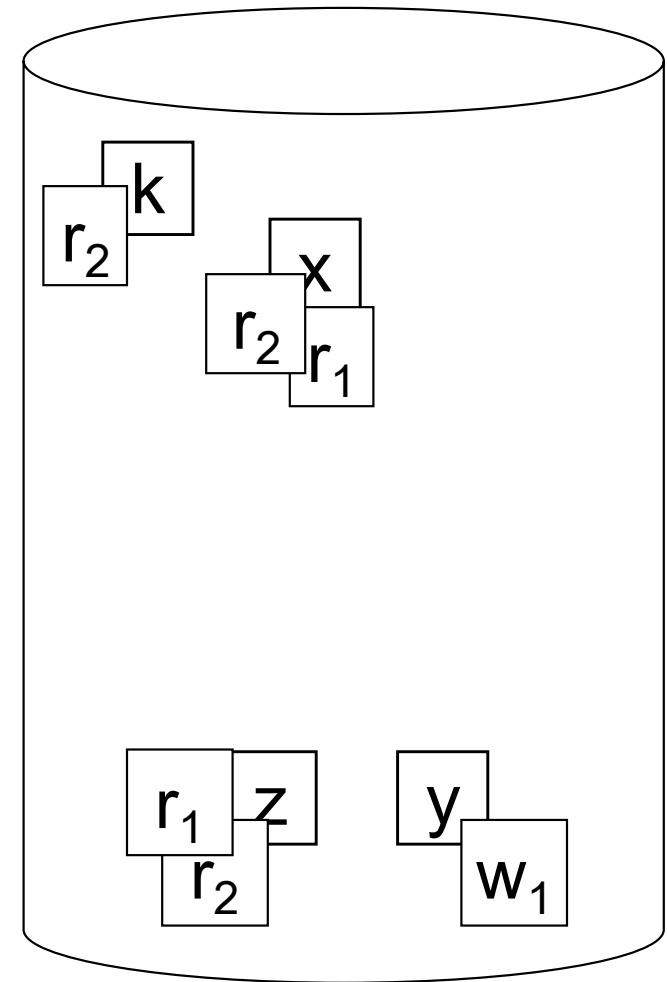
- Consider the current set of partially executed open transactions for each moment in time.
- **Simple** scheduler: Block any operation that would cause this set to have a **shared object**.
- **Common** scheduler: Block any operation that would cause this set to have a **conflict**.
- Can all operations on the right pass in common scheduler?



Only in common scheduler

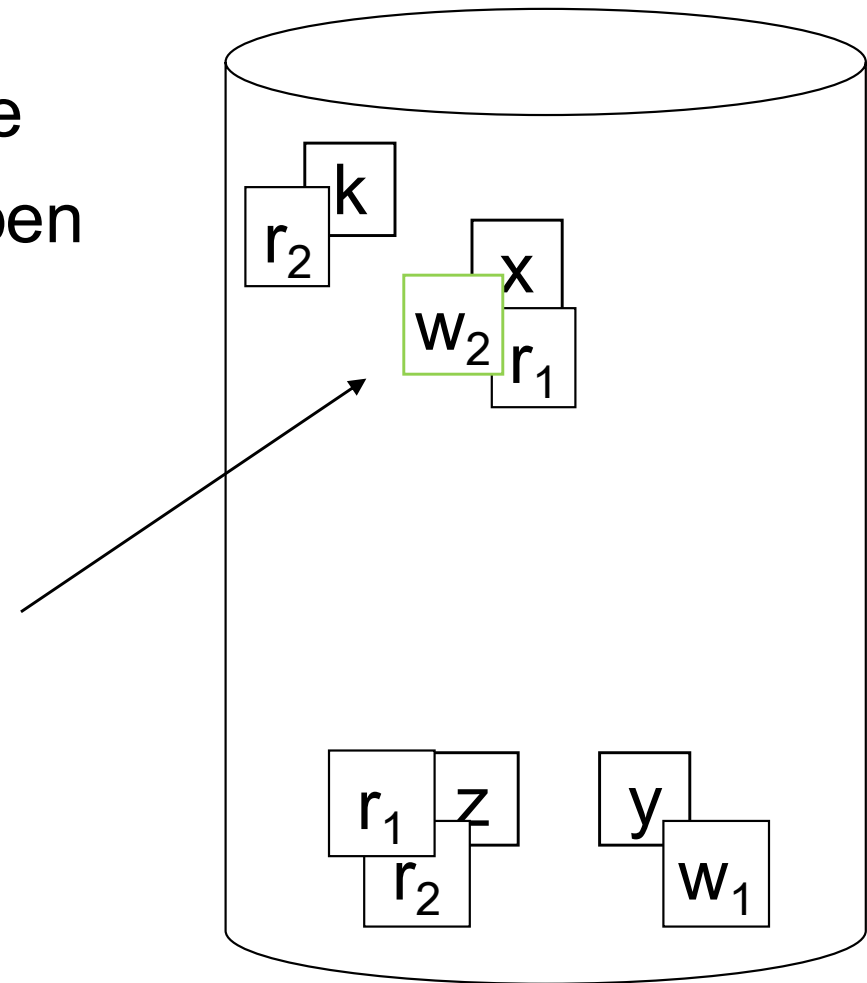
Operation tags are natural

- Schedulers are usually defined directly with locks instead of our tags.
- Advantage of *declarative* definition with tags:
- We do **not** say **how** and when to place and remove locks (imperative)
- We say **what** should be allowed: all operations of ongoing transactions (declarative).
- An observable concept



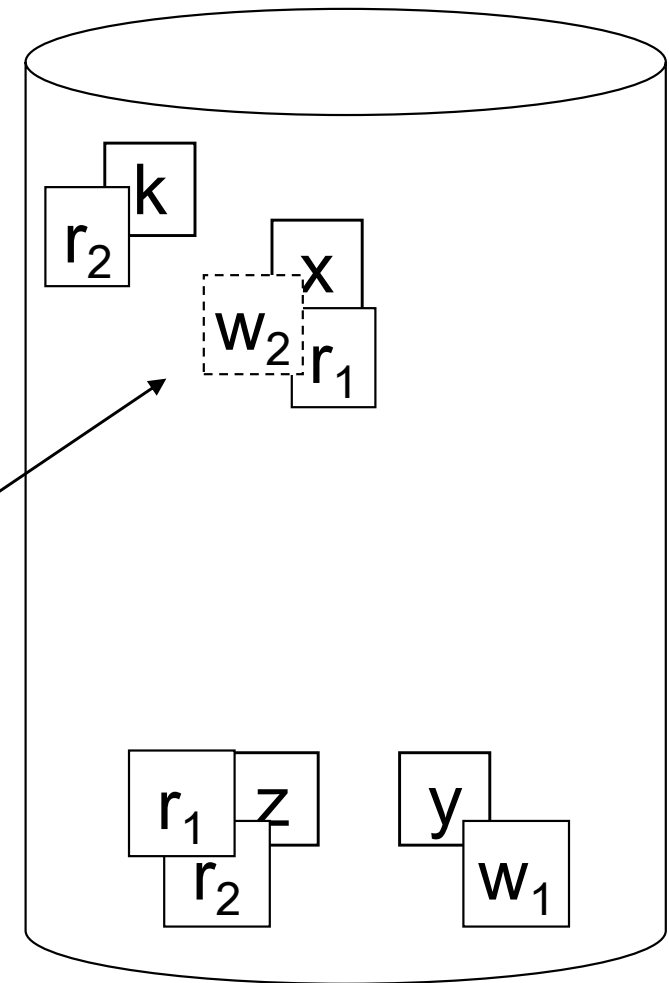
Declarative definition of schedulers

- **Common scheduler:** Block any operation that would cause the current set of partially executed open transactions to have a **conflict**.
- Example:
- Under this rule: is w_2 on the right blocked?



Declarative definition of schedulers

- **Common scheduler**: Block any operation that would cause the current set of partially executed open transactions to have a **conflict**.
- Example:
- Under this rule: is w_2 on the right blocked? **Yes!**
- (We will depict blocked operations with dotted outlines).
- After a transaction finishes: some blocked transactions may continue.



The common scheduler with upgrade locks

- Akin to schedulers used in practice.
- uses several kinds of locks:
 - read locks, also known as shared locks (S locks)
 - Several transactions can have a read lock on the same object:
 - an object with read locks can only be read, not written.
 - Advantage: **conflict-free transactions are not delayed.**
 - write locks, also known as exclusive locks (X locks)
 - If an object has a write lock on x, no other lock can be set on x.
 - The owner can read and write the object.
 - A lone read lock on x can be *upgraded* to a write lock by owner.
 - upgrade locks (a.k.a. update locks , U locks)
 - Help in acquiring write locks in certain conditions

Scheduling with shared locks

- S: $r_1[x], r_2[x], c_2, r_1[y], r_3[z], w_1[y], c_1, w_3[x], c_3$
 - TA1: $r_1[x], r_1[y], w_1[y], c_1$
 - TA2: $r_2[x], c_2$
 - TA3: $r_3[z], w_3[x], c_3$
-

transaction TA2 not waiting any more.

transaction TA3 is waiting, cannot acquire write lock

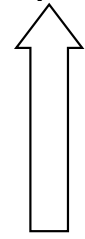
Problem with read and write locks:

- As long as several transactions have a read lock on the same object, a write lock cannot be acquired. Writing transactions have to wait.
- Without further precautions, a writing transaction might never be able to acquire the write lock, because new transactions continuously start to read: the writing transaction would be in a *live-lock* (a form of *starvation*):
 - S: $r_1[x], r_2[x], r_3[x], c_2, r_4[x], r_5[x], c_3, c_4, r_6[x], \dots$
 - TA1: $r_1[x], w_1[x]$ _____?

Solution: upgrade locks

- The first writing transaction gets the third type of lock, the **upgrade lock** (U lock):
 - No new reader can access this object, before the write was de-queued and executed.
 - Once all readers have finished, the writing transaction gets the exclusive lock:

◦ S: $r_1[x], r_2[x], r_3[x], c_2,$ $c_3, w_1[x], c_1, r_4[x], \dots$

◦ TA1: $r_1[x],$ $w_1[x]$ , c_1 

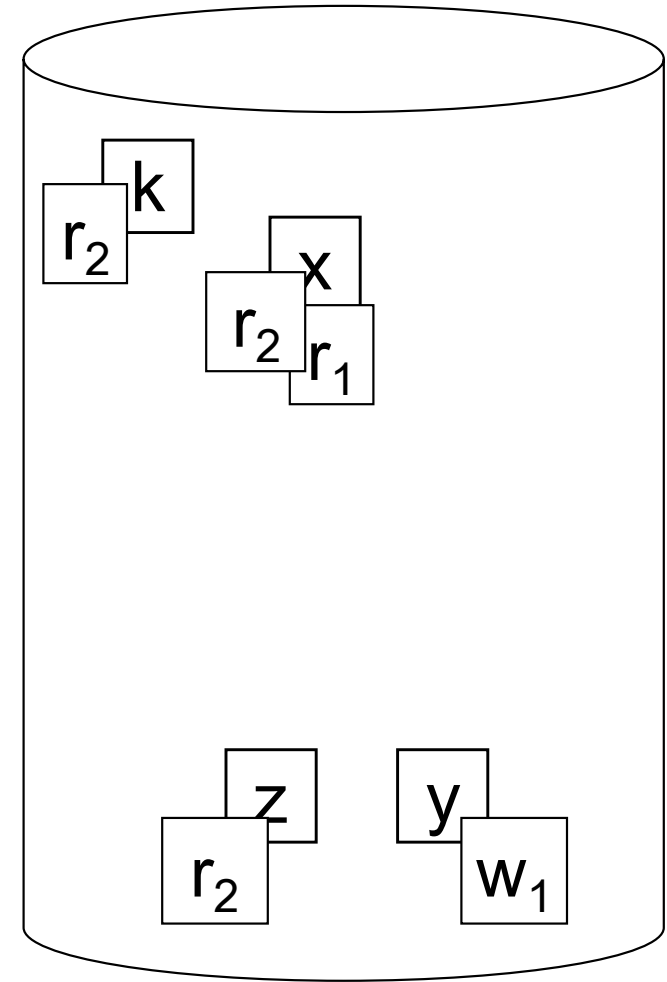
◦ TA4: $r_4[x]$ , $\dots$

transaction TA1 waiting, but acquires U lock.

transaction TA4 is waiting, cannot acquire read lock

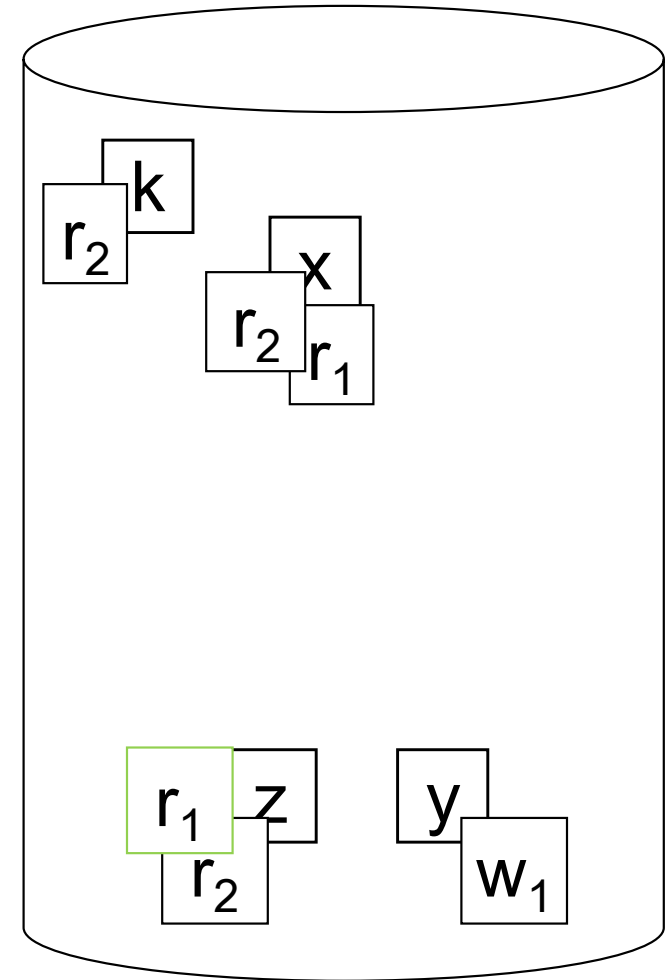
Locks can be computed from current ops

- All three locks can be computed from the list of operations present at the object.
- In principle no locks as separate datastructure necessary.
- Still locks can be a helpful way to talk about current access to objects.



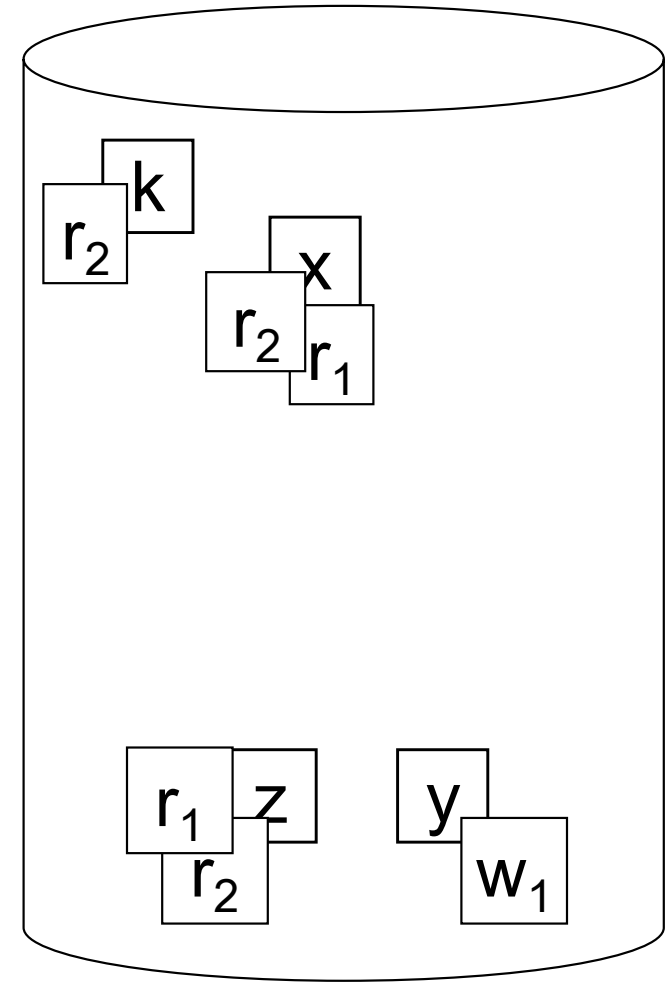
Locks can be computed from current ops

- All three locks can be computed from the list of operations present at the object.
- In principle no locks as separate datastructure necessary.
- Still locks can be a helpful way to talk about current access to objects.



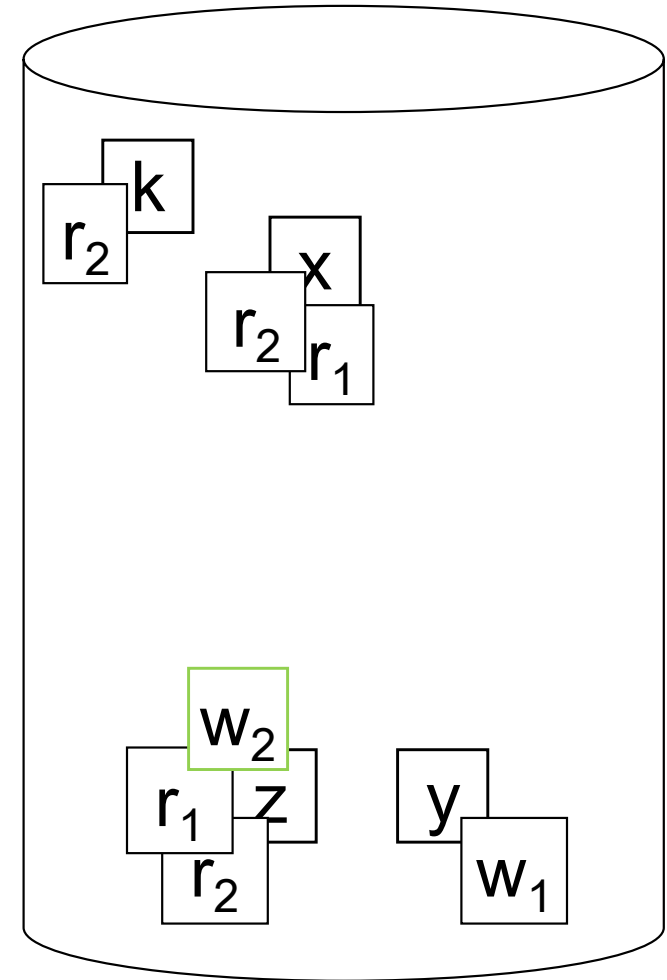
Locks can be computed from current ops

- All three locks can be computed from the list of operations present at the object.
- In principle no locks as separate datastructure necessary.
- Still locks can be a helpful way to talk about current access to objects.



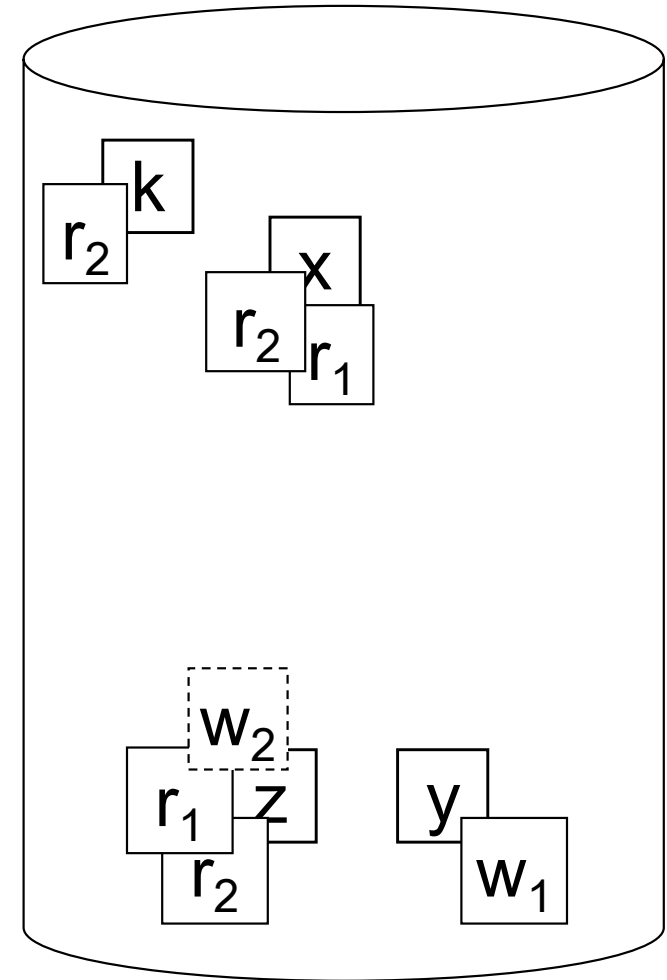
Locks can be computed from current ops

- All three locks can be computed from the list of operations present at the object.
- In principle no locks as separate datastructure necessary.
- Still locks can be a helpful way to talk about current access to objects.



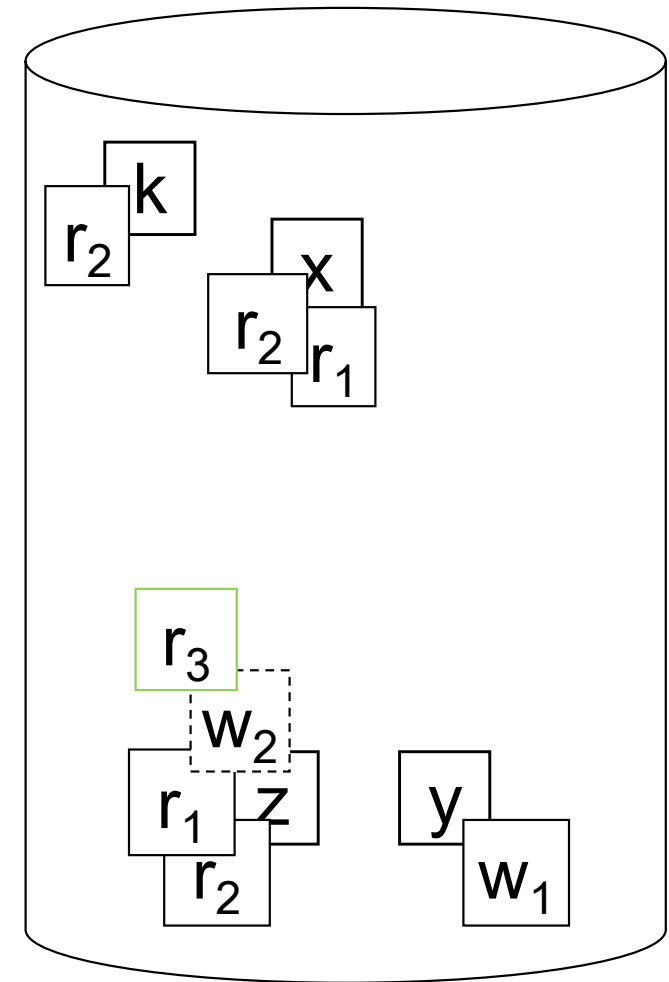
Locks can be computed from current ops

- All three locks can be computed from the list of operations present at the object.
- In principle no locks as separate datastructure necessary.
- Still locks can be a helpful way to talk about current access to objects.



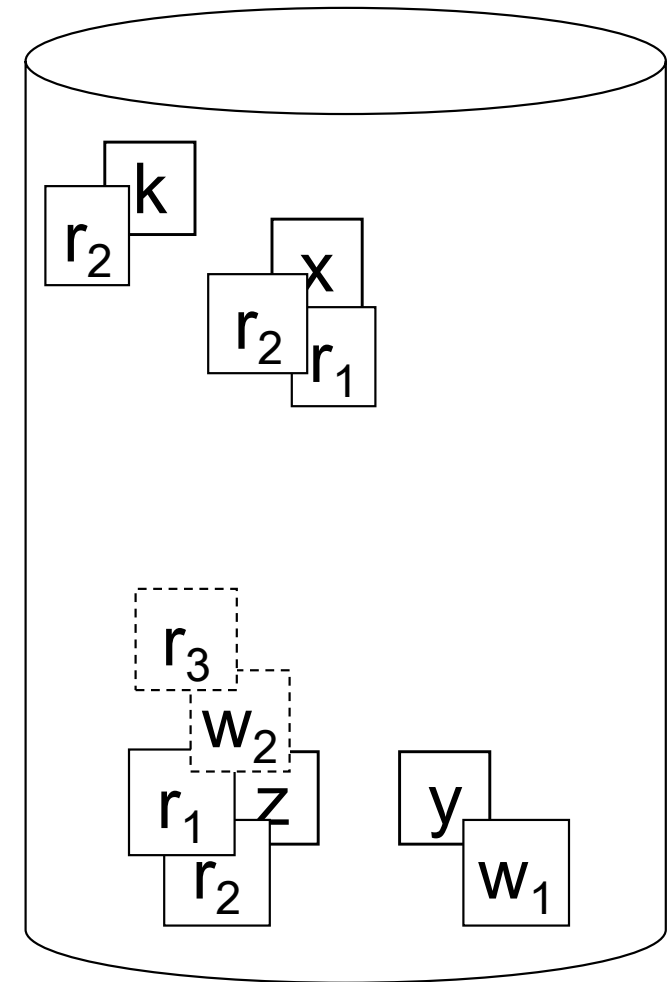
Locks can be computed from current ops

- All three locks can be computed from the list of operations present at the object.
- In principle no locks as separate datastructure necessary.
- Still locks can be a helpful way to talk about current access to objects.



Locks can be computed from current ops

- All three locks can be computed from the list of operations present at the object.
- In principle no locks as separate datastructure necessary.
- Still locks can be a helpful way to talk about current access to objects.



Upgrade locks continued

- Only one transaction can acquire an U lock
- Later transactions that try to do so will be blocked
- Often expressed in a matrix:
 - the columns denote the highest lock owned by another transaction that is already present at this object.
 - the rows represent the lock that a transaction wants to acquire.
 - Note: the view here is that a write requests both and U and X lock.

lock requested

	highest lock present		
	S	U	X
S	y	n	n
U	y	n	n
X	n	n	n

y: lock granted and
transaction
not blocked

n: lock not yet
granted
and transaction
blocked

ACID Isolation

- Database operations in a transaction appear isolated from database operations of all other transactions
 - does not apply to non-database operations of scripts
- Isolation is expensive:
- May lead to delays, deadlocks, and/or aborts.
 - and can therefore be relaxed: *isolation levels*.
- Highest isolation level:
- Transactions must have same view as with DB-wide MutEx.
- All committed transactions must have the same effect on the database as the execution of the transactions with Mutual exclusion in some order: [Serializability](#).

Summary

- Conflict-free transactions fulfil ACID isolation.
- The simple scheduler delays **conflict-free** transactions, is pessimistic.
- The common scheduler
 - plausibly **locks less** often in practice than the simple scheduler
 - needs a more elaborate locking protocol.
- Locks can be inferred from the operations present; only for upgrade locks we need to consider a blocked operation.